

K-2SO Animatronic Controller

Systemdokumentation v1.3.0

Vollstaendiges Nutzer-Handbuch fuer den K-2SO Animatronic Controller. Aufbau, Konfiguration, Kalibrierung, Aufnahme, Wiedergabe -- alles in einem.

Version 1.3.0 -- 2026-04

Printed-Droid.com

<https://www.printed-droid.com>

Haftungsausschluss

K-2SO Animatronic Controller ist ein DIY-Projekt. Aufbau, Verkabelung, Konfiguration und Betrieb eines Animatronic-Systems umfassen Arbeiten, die bei unsachgemaesser Ausfuehrung Sach- oder Personenschaden verursachen koennen -- insbesondere durch Servos unter Last, heisse Loetgeraete, netzversorgte Netzteile und Akkupacks.

Mit der Nutzung dieser Dokumentation, Hard- und Software erkennst du an:

- Du bist alleine verantwortlich fuer alle Arbeiten an deinem Build.
- Du verfuegst ueber Grundkenntnisse in Elektronik und Arduino-Tooling, oder holst dir qualifizierte Hilfe.
- Du pruefst Spannung, Polung, Sicherungen und Steckverbindungen vor jedem Einschalten.
- Du verwendest nur passende Komponenten (5V/3A PSU Minimum, abgesicherte Versorgungsleitungen).
- Du beachtest die oertlichen Elektrovorschriften.
- Du aenderst das Default-Webpasswort, bevor das Geraet im Netz erreichbar ist.

Der Autor und alle Beteiligten uebernehmen keinerlei Haftung fuer Schaeden durch falsche Verkabelung, ungeeignete Komponenten, unsachgemaesse Bedienung oder unzureichende Sicherheitsmassnahmen.

Die Dokumentation und Software werden **"wie besehen"** ohne jede Garantie bereitgestellt. Im Zweifel qualifizierte Hilfe einholen, bevor am System gearbeitet wird.

Inhaltsverzeichnis

Haftungsausschluss	2
Inhaltsverzeichnis	3
Einfuehrung -- Was ist der K-2SO Animatronic Controller?	8
Versions-Uebersicht	9
Hauptfunktionen (v1.3.0)	9
Zielgruppe dieses Handbuchs	9
Sicherheit	10
Stromversorgung	10
Servo-Quetschgefahr	10
Montage im Droiden	10
Netzwerksicherheit	10
Schnell-Checkliste Sicherheit	11
Schnelleinstieg	12
Sketch hochladen	14
Toolchain-Optionen	14
Benoetigte Libraries (v1.3.0)	14
Board-Einstellungen (Arduino IDE)	14
USB / Treiber-Probleme	14
Haeufige Compile-Fehler	15
WiFi & Web-UI	16
Verbindung im STA-Modus	16
AP-Fallback	16
WebUI erreichen	16
Authentifizierung	16
WebUI-Rundgang	16
Board & Pin-Belegung	21
GPIO-Pin-Belegung (v1.3.0)	21

Steckergruppen	22
Verdrahtungs-Hinweise	23
Stromversorgung & Netzteil	24
Strombudget	24
Empfohlene Topologie	24
Akku-Builds	24
LED-Hardware	25
Eye-NeoPixels: 7-LED vs 13-LED	25
Detail-LED-Strip	25
Status-LED	25
Servo-Kalibrierung	28
Default-Werte	28
Kalibrierungs-Workflow	28
Profile	29
Eye-Animationen	30
Animation-Modes	30
Farbe & Helligkeit	30
Audio (DFPlayer Mini)	31
SD-Card-Vorbereitung	31
CLI-Referenz	31
Default-Lautstaerke & Auto-Mode	31
IR-Fernbedienung einrichten	32
Learning-Mode-Workflow	32
IR-Befehle	32
Personality-Modi	34
Mode-Verhalten anpassen	34
Wake- / Sleep-Verhalten	34
Sequence-Recording (NEU v1.3.0)	35
Was wird pro Frame aufgenommen	35

Kapazitaet	35
Aufnahme-Workflow (Serial-CLI)	35
Wiedergabe-Befehle	36
Crash-sicheres Speichern (NEU v1.3.0 Follow-up)	37
Pause und Resume (nur Web UI)	37
Playlist-Chaining (NEU v1.3.0)	39
Playlist aufbauen	39
Verhalten und Robustheit	39
Beispiel-Performance	40
IR-zu-Sequenz-Mapping (NEU v1.3.0)	42
Voraussetzungen	42
Mapping-Befehle	42
Beispiel	42
Serial-CLI-Referenz	45
Top-Level-Befehle	45
Hierarchische Befehle	45
Haeufige Schnell-Befehle	46
Voice-Assistants	47
Trigger-Endpunkte	47
Setup-Beispiel: Google Home / IFTTT	47
Curl / Manueller Test	47
Konfiguration & Backup	48
Was wird gespeichert	48
Backup ueber Serial	48
Restore ueber Serial	48
Profile vs Backup	49
Migration v1.2.x auf v1.3.0	50
Was passiert beim ersten Boot nach Upgrade	50
Erhaltene Felder	50
Neue / zurueckgesetzte Felder	50

Empfohlener Upgrade-Ablauf 50

Fehlersuche 52

Symptom-zu-Fix-Tabelle 52

Versions-Historie 53

v1.3.0 -- Promotion-Kandidat (2026-04) 53

v1.2.4 -- Vorherige Stable (2026-04) 54

v1.2.5 -- Beta (2026-04) 54

v1.2.3 -- Stable (2026-03) 54

v1.1.0 -- Archive (2026-02) 54

Teil I

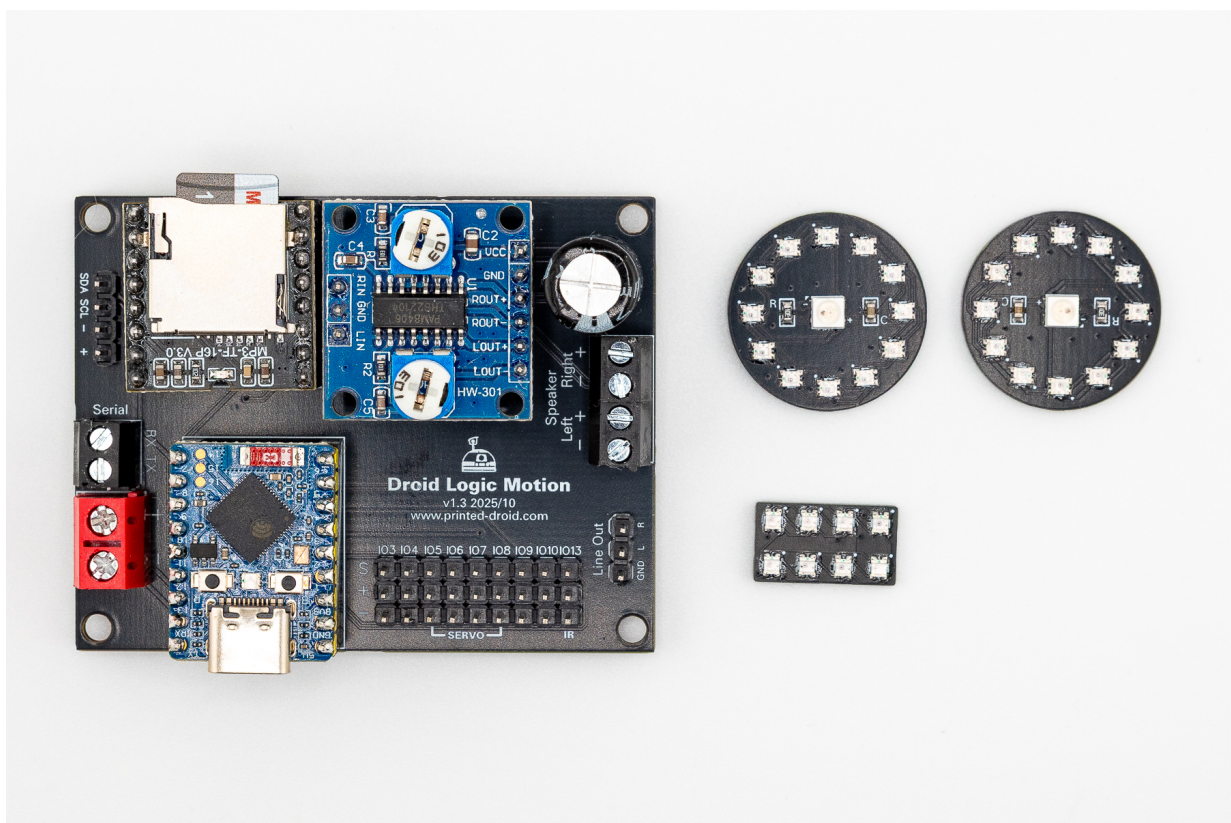
Erste Schritte

Einfuehrung -- Was ist der K-2SO Animatronic Controller?

Der K-2SO Animatronic Controller ist ein ESP32-S3-basierter Controller fuer den K-2SO-Animatronic von **Droid Division** (dort als *Security Droid* gefuehrt). Er steuert vier Servos fuer Augen- und Kopfbewegung, zwei NeoPixel-Augenringe, einen Detail-LED-Strip, eine Status-LED, einen DFPlayer Mini fuer Audio, einen IR-Empfaenger, ein WiFi-Webinterface und persistente EEPROM-Konfiguration. Seit v1.3.0 kann er zusaetzlich eigene Bewegungssequenzen aufnehmen, abspielen und verketten, die in LittleFS gespeichert werden.

Wo gibt es den Droiden: der K-2SO-Bausatz und fertige Figuren werden von Droid Division verkauft -- <https://www.droiddivision.com/>. Die Droid-Division-eigene Builder-Community findet sich auf Facebook: <https://www.facebook.com/groups/2505708886350784/>.

Controller-Firmware und Support: Quellcode, Releases und Issue-Tracker fuer diesen Controller liegen auf GitHub: <https://github.com/PrintedDroid/K-2SO-Animatronic>. Builder-Fragen zum Controller und zum Droid-Logic-Motion-Board werden in der Printed-Droid-Facebook-Gruppe beantwortet: <https://www.facebook.com/groups/printeddroid/>.



Droid-Logic-Motion-v1.3-Controller mit Eye-Boards und Detail-LED-Board.

Versions-Uebersicht

Version	Status	Highlights
v1.1.0	Archive (nur Lesezugriff)	Erstausgabe: Grund-Servosteuerung, einfache Eye-Animation, DFPlayer-Audio, rudimentaere Serial-CLI.
v1.2.4	Vorherige Stable	Stabiler Patch auf v1.2.3 mit 6 kritischen und High-Severity-Stabilitaetsfixes (non-blocking Status-LED, restore-Checksum-Verify, Profile-Slot-Wache, Servo-Limit min<max, blocking-CLI-Timeout, Detail-LED-Loop-Wache). EEPROM-kompatibel zu v1.2.3.
v1.3.0 (dieses Handbuch)	Stable-Kandidat	Promoviert die v1.2.5-Beta-Features zu Stable: Frame-basiertes Sequence-Recording, Playlist-Chaining, IR-Button-zu-Sequenz-Mapping. Bringt zusaetzlich automatische v1 -> v2 EEPROM-Migration, die alle bestehenden Daten erhaelt.

Hauptfunktionen (v1.3.0)

- **4-Servo-Motion:** Eye Pan/Tilt + Head Pan/Tilt, kalibrierbare Center und Limits, 5 Profile.
- **Dual NeoPixel Eyes:** 14 Animation-Modes, runtime-umschaltbar 7-LED oder 13-LED.
- **WS2812 Detail-Strip:** 1-8 LEDs auf GP10, mehrere Muster, Helligkeit einstellbar.
- **Status-LED:** Einzel-WS2812 auf GP21 -- zeigt Boot/WiFi/Modus/Fehler-Zustand auf einen Blick.
- **DFPlayer-Audio:** 40+ K-2SO-Soundclips, Lautstaerke einstellbar, Folder/Track-Adressierung.
- **IR-Fernbedienung:** 21-Button-Lernfernbedienung, NEC/Sony-Codecs, Button-zu-Sequenz-Mapping.
- **WiFi-WebUI:** STA + AP-Fallback (K2SO-XXXXXX), HTTP Basic Auth, mDNS k2so.local.
- **Voice-Assistants:** 10 Trigger-Endpunkte fuer Google Home / Alexa / Siri.
- **Serial-CLI:** 115200 Baud, ~30 Root-Befehle mit Subcommands, Hilfetexte, Status, Monitor.
- **Sequence-Recording (NEU v1.3.0):** Frame-basiert, bis 200 Frames, JSON-Speicher auf LittleFS.
- **Playlist-Chaining (NEU v1.3.0):** Bis zu 10 Sequenzen verketten, optional Loop.
- **IR-zu-Sequenz-Mapping (NEU v1.3.0):** Jeden der 21 IR-Buttons mit gespeicherter Sequenz belegen.
- **EEPROM-Auto-Migration (NEU v1.3.0):** v1-Layout (v1.2.3 / v1.2.4) -> v2 ohne Datenverlust.

Zielgruppe dieses Handbuchs

Dieses Handbuch richtet sich an **Builder**, die die Hardware bereits aufgebaut haben und das System bedienen, konfigurieren und erweitern wollen. Etwas Erfahrung mit Arduino IDE oder PlatformIO wird vorausgesetzt. Die **Developer Reference** (separates PDF) deckt EEPROM-Struct-Details, HTTP-API-Endpunkte und Library-Versionseinschraenkungen ab.

Tipp: Lies das Sicherheitskapitel bevor du das erste Mal einschaltest. Ein 5V/3A-Netzteil ist das Minimum -- schwaechere Versorgungen fuehren zu Brownouts beim DFPlayer + WiFi-Boot.

Sicherheit

Dieses Kapitel listet die praktischen Sicherheitsregeln fuer Bau, Verkabelung und Betrieb der Animatronic. Der vollstaendige Haftungsausschluss steht auf Seite 2.

Stromversorgung

5V bei 3A (15W) ist das absolute Minimum. Vier Servos koennen kurzzeitig 1,5-2A gleichzeitig ziehen, wenn sie unter Last die Richtung wechseln. Plus ESP32-S3 (~150 mA-Peaks bei WiFi-Anmeldung), DFPlayer (250 mA beim Track-Start) und vier NeoPixel-Strings auf Maximalhelligkeit -- das 3A-Budget ist realistisch, nicht grosszuegig.

Brownout-Warnung: Die Kombination DFPlayer + WiFi-Boot ist stromhungrig. Bei zu schwachem Netzteil sieht man im Serial-Log einen ESP32-Reset mit "Brownout detector was triggered". Dann ein staerkeres Netzteil verwenden oder Servos und Logik separat versorgen, auf gemeinsame Masse achten.

Servo-Quetschgefahr

Servos sind nicht zimperlich. Beim ersten servo test all oder nach Aenderung der Limits: Finger, Kabel und Deko von Servohebeln und mechanischen Anlenkungen fernhalten. Den Test mit abgenommenem Kopf machen, bis du den Limits vertraust.

Montage im Droiden

Der K-2SO von Droid Division ist eine freistehende, lebensgrosse Figur. Die fertige Kopf-Baugruppe (Augen, NeoPixel-Boards, Mechanik, Servos) wiegt trotzdem ein paar hundert Gramm. Servos verhalten sich unter Last anders als auf dem Werk Tisch: Haltedrehmoment gegen Schwerkraft kommt schneller, Getriebe verschleissen frueher. Nach Einbau von Controller und LED-Boards in den Kopf: Servo-Limits neu pruefen, neu zentrieren und mindestens eine Minute pro Personality-Mode beobachten, bevor der Droid ins Display geht.

Netzwerksicherheit

Das Webinterface ist mit HTTP Basic Auth geschuetzt -- Default-Zugang admin / k2so2025. **Passwort aendern**, bevor der Droid an einem Netzwerk ausserhalb der Werkstatt haengt. Sonst kann jeder im selben WiFi die Steuerung erreichen.

Schnell-Checkliste Sicherheit

#	Pruefung	Warum
1	5V/3A-Netzteil Minimum, abgesicherte Leitungen	Brownouts killen DFPlayer-UART und WiFi-Anmeldung.
2	Polung und Pin-Belegung vor erstem Einschalten pruefen	GPIO-Kurzschluss auf 5V zerstört den ESP32-S3.
3	Servos frei beweglich, keine Last beim ersten servo test all	Vermeidet abgenudelte Getriebe falls ein Limit falsch ist.
4	k2so2025-Default-Passwort vor Netzbetrieb aendern	Netzbetrieb mit Default-Passwort = offener Zugang.
5	Hauptschalter auf der 5V-Schiene einbauen	Du willst den Droid mehrmals schnell stromlos schalten koennen.

Schnelleinstieg

Dieses Kapitel bringt einen frisch geflashten v1.3.0-Controller in 15-20 Minuten vom kalten Zustand auf "Droid bewegt sich und spricht", angenommen die Hardware ist verkabelt und eine microSD mit Sounds steckt im DFPlayer. Wenn unterwegs etwas klemmt: ins Troubleshooting-Kapitel springen.

Den ersten Servo-Test mit abgenommenem Kopf machen. Falsche Limits fahren Servos in mechanische Anschlaege; die Getriebe ueberleben das unter Last nicht.

1. Arduino-Libraries installieren

Arduino IDE oeffnen, Library Manager. Installieren: **Adafruit NeoPixel**, **ESP32Servo**, **IRremote**, **DFRobotDFPlayerMini**, **ArduinoJson** (v7). LittleFS kommt mit dem ESP32-Core.

2. ESP32-Board-Package hinzufuegen

Werkzeuge → Board → Boardverwalter. "esp32" suchen, **esp32 by Espressif Systems** (3.x) installieren. Dann Werkzeuge → Board → ESP32 → **Waveshare ESP32-S3-Zero**. FQBN: esp32:esp32:waveshare_esp32_s3_zero. COM-Port waehlen.

3. WiFi-Credentials setzen

K_2SO_DroidLogicMotion_v1.3.0/config.h oeffnen, WIFI_SSID und WIFI_PASSWORD anpassen. Platzhalter koennen auch bleiben -- der Controller startet den K2SO-XXXXXX-AP-Fallback, du konfigurierst WiFi spaeter im Browser.

4. Upload

Sketch hochladen. Erster Upload dauert etwas, weil die ESP32-Toolchain kalt ist. Der Serial Monitor bei 115200 Baud sollte zeigen: `=== K-2SO Animatronic Controller v1.3.0 ===`.

5. Boot-Phasen beobachten

Status-LED-Ueborgaenge: blau blinkend (Boot) → gelb blinkend (WiFi-Verbindung) → gruen 2s solid (WiFi up) → Mode-Farbe (Default amber bei IDLE / eisblau bei SCANNING). Bei mehr als 30 s gelb: AP-Fallback ist aktiv.

6. Controller im Netz finden

STA: `http://k2so.local` oder die IP aus dem Serial-Log. AP-Fallback: Handy mit K2SO-XXXXXX verbinden (Passwort k2so2025), dann `http://192.168.4.1`.

7. Einloggen

Default admin / k2so2025. **Passwort aendern, bevor der Droid in ein echtes Netz geht.**

8. Erster Servo-Test

`servo test all`. Die vier Servos sollten nacheinander ihre Limits abfahren. Wenn ein Servo gegen einen Anschlag faehrt oder brummt: Droid stoppen, Limits anpassen (siehe Servo-Kalibrierung).

9. Erster Eye-Test

`led mode pulse` -- beide Augen sollten weich pulsieren. `led color 0 0 255` setzt Blau. `led mode scanner` zeigt den klassischen Scanner-Ring.

10. Erster Sound-Test

`sound play 1 1` spielt Ordner 1, Track 1. Wenn stumm: SD-Card-Layout pruefen (siehe Audio-Kapitel). "DFPlayer-not-ready" beim Boot deutet auch auf SD-Probleme hin.

11. Optional: erste Sequenz aufnehmen

WebUI oeffnen, Sequence Editor, **Record**, Name vergeben (z.B. begruessung), Gamepad-Slider in Pose, **Add Frame** mit Dauer. Wiederholen. Speichern. **Play**. Voller Workflow im Kapitel Sequence Recording.

Tipp: wenn die Boot-Meldungen zu schnell durchrauschen: `help` oder `status` jederzeit eintippen fuer CLI-Zusammenfassung bzw. aktuellen Zustand.

Sketch hochladen

Dieses Kapitel geht tiefer als der Schnelleinstieg: welche IDE, welches Board-Package, welche Libraries in welchen Versionen, haeufige Fehler mit USB / Treibern / Partition-Layout und wie das v1.3.0-Groessenbudget gegenueber fruheren Releases liegt.

Toolchain-Optionen

Zwei Wege werden unterstuetzt: **Arduino IDE 2.x** (einfachster Erstkontakt) und **PlatformIO** (besser fuer reproduzierbare Builds, Multi-Versionshandhabung). Beide produzieren dieselbe Firmware -- der Sketch in K_2SO_DroidLogicMotion_v1.3.0/ kompiliert in beiden.

Benoetigte Libraries (v1.3.0)

Library	Empfohlene Version	Notes
Arduino ESP32 core	3.x	Espressif Systems, via Boards Manager. Provides the Waveshare ESP32-S3-Zero target.
Adafruit NeoPixel	latest 1.x	RMT-Treiber kompatibel mit IRremote. Getestet: 1.15.x.
ESP32Servo	latest 1.x	LEDC channels 0-3 reserved for the four servos.
IRremote	latest 4.x	21-button learning, NEC/Sony codecs.
DFRobotDFPlayerMini	latest 1.x	UART @ 9600 baud on GP11/GP12.
ArduinoJson	7.x	WebUI JSON serialization + sequence file format.
LittleFS	kommt mit ESP32-Core	Sequence storage on the flash partition.

Library-Versionen bewusst waehlen. Alte NeoPixel + neue IRremote (oder umgekehrt) koennen auf RMT-Kanaele kollidieren. Die obige Kombination ist getestet; wenn du eine aenderst: erneut testen.

Board-Einstellungen (Arduino IDE)

Nach Installation des ESP32-Cores: **ESP32** → **Waveshare ESP32-S3-Zero** waehlen. Defaults reichen; relevant: USB CDC On Boot = Enabled, Flash Size = 4MB, Partition Scheme = **No OTA (2MB APP / 2MB SPIFFS)**, CPU Frequency = 240 MHz. Mit diesem Partitions-Schema liegt der v1.3.0-Sketch bei ~60% Flash (1,27 MB / 2 MB) und ~16% RAM. Die 2-MB-SPIFFS-/LittleFS-Haelfte wird fuer Sequence- und Playlist-Storage genutzt.

USB / Treiber-Probleme

Der ESP32-S3-Zero meldet sich als USB-CDC-ACM-Geraet. Unter Windows ggf. **CH340-** oder **CP210x-**VCP-Treiber installieren, je nach USB-Seriell-Bridge auf dem Board. macOS und Linux funktionieren meist out of the box. Wenn der Upload "no port found" meldet: BOOT-Taster gedruickt halten beim USB-Einstecken -- der Chip geht in den Bootloader-Modus, unabhaengig von der laufenden Firmware.

Haeufige Compile-Fehler

Error	Ursache	Fix
ArduinoJson.h not found	Library fehlt oder v6 installiert	Library Manager: install ArduinoJson 7.x.
PWM channel taken	NeoPixel hat LEDC statt RMT belegt	Adafruit NeoPixel 1.15.x oder neuer (RMT-Treiber).
cannot find esp32:esp32	Board-Manager-URL fehlt	In IDE-Einstellungen https://espressif.github.io/arduino-esp32/package_esp32_index.json eintragen.
Sketch too big	Falsches Partitions-Schema	Werkzeuge → Partition Scheme → "No OTA (2MB APP / 2MB SPIFFS)" . Der v1.3.0-Sketch nennt das im Header als empfohlenes Schema. Default 4MB with spiiffs geht auch, laesst aber nur ~70 KB frei.

Tipp: der v1.3.0-Sketch nennt *No OTA (2MB APP / 2MB SPIFFS)* im Header als empfohlenes Partitions-Schema. Damit liegt der Build bei ~60% Flash (1,27 MB / 2 MB) -- viel Headroom fuer kuenftige Features, ohne LittleFS-Sequence-Storage aufzugeben.

WiFi & Web-UI

Der Controller bietet zwei WiFi-Modi: **STA** (verbindet sich mit deinem Heimnetz) und **AP-Fallback** (oeffnet einen eigenen Access Point K2S0-XXXXXX, wenn STA fehlschlaegt). Das WebUI ist das freundliche Gegenstueck zur Serial-CLI -- alles, was du seriell machen kannst, geht auch im Browser, plus Gamepad-Servosteuerung, Sequence-Editor und IR-Map-Editor.

Verbindung im STA-Modus

(a) Compile-Time: WIFI_SSID und WIFI_PASSWORD in config.h vor dem Flashen anpassen. **(b) Runtime:** nach AP-Fallback verbinden, ueber WebUI-Settings setzen, oder Serial-CLI: `wifi set MyNet MyPass + wifi reconnect`.

AP-Fallback

Wenn der Controller das konfigurierte WiFi binnen 30 s nicht erreicht (oder keine Credentials gesetzt sind), startet er einen eigenen AP. Die SSID kommt aus der MAC: K2S0-AABBCC. Default-Passwort k2so2025. Verbinden, dann `http://192.168.4.1`. Hier kannst du WiFi-Credentials setzen und in den STA-Modus rebooten. AP-Fallback laesst sich mit `ap disable` abschalten.

WebUI erreichen

Mode	URL	Hinweise
STA	<code>http://k2so.local</code>	mDNS, funktioniert in den meisten Heimnetzen. Manche Router blocken mDNS -- dann die IP aus dem Serial-Log.
STA	<code>http://<ip></code>	Direkte IP. Anzeigen mit <code>status</code> oder <code>wifi show</code> .
AP	<code>http://192.168.4.1</code>	Always-on IP for the AP fallback.

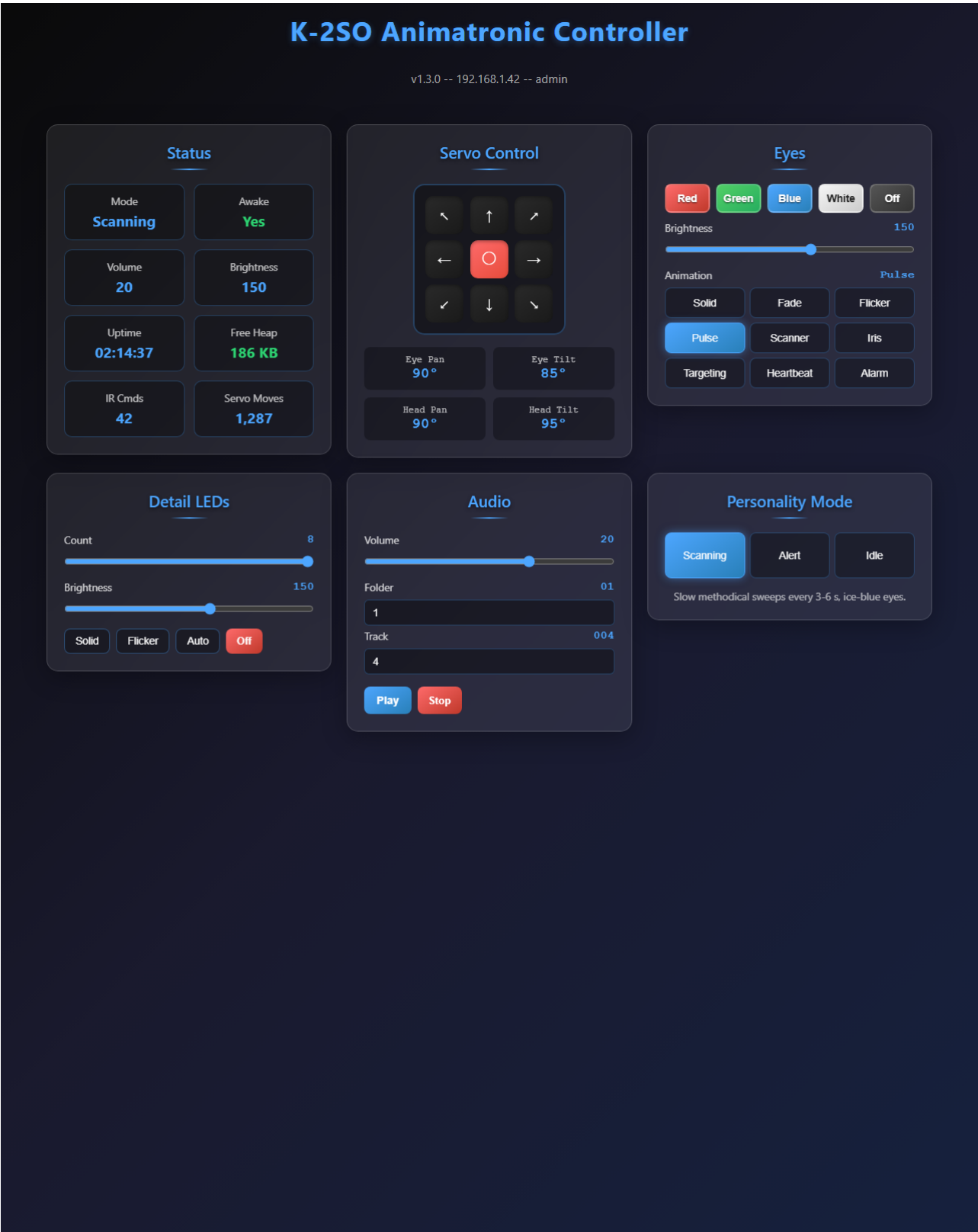
Authentifizierung

Alle WebUI-Seiten und Trigger-Endpunkte sind durch HTTP Basic Auth geschuetzt. Default `admin / k2so2025`. In config.h aendern, bevor der Droid an ein Netzwerk geht.

WebUI-Rundgang

- **Status:** Modus, Awake-Zustand, Lautstaerke, Helligkeit, Uptime, freier Heap, IR-Befehlszaehler, Servo-Bewegungszaehler.
- **Servos (Gamepad):** zwei virtuelle Sticks fuer Eye Pan/Tilt und Head Pan/Tilt; Werte werden auf die konfigurierten Limits geklemmt.
- **Augen:** 14 Animation-Modes als Buttons; Color-Picker; Helligkeits-Slider; *red / green / blue / white / off* Shortcuts.
- **Detail-LEDs:** Anzahl-Auswahl (1-8), Muster-Auswahl, Color-Picker, Helligkeits-Slider, on/off, Auto-Mode.
- **Audio:** Lautstaerke-Slider, Folder/Track-Eingabe, Stop-Button.

- **Modus:** Scanning / Alert / Idle Radio-Buttons.
- **Sequenzen (NEU v1.3.0):** Liste gespeicherter Sequenzen, Play/Loop/Stop/Pause/Resume/Delete pro Eintrag, "Record"-Form, Frame-Editor.
- **Playlist (NEU v1.3.0):** add/clear/play/loop, aktuelle Playlist-Anzeige.
- **IR-Mapping (NEU v1.3.0):** Tabelle der 21 Buttons mit Dropdown zur Sequenz-Zuordnung.
- **Einstellungen:** WiFi, AP, Passwort, Helligkeits-Defaults, Eye-Hardware-Version (7-LED / 13-LED).



Web-UI-Dashboard (v1.3.0).

K-2SO Animatronic Controller

v1.3.0 -- 192.168.1.42 -- admin

Sequence Editor

● Recording: **greeting_v2** - Frame 4 captured

greeting_v2

1500

+ Add Frame

Save

Cancel

Frames (4)

1	EP=90 ET=90 HP=90 HT=90	eye=pulse #007FFF	sound=--	1000 ms
2	EP=90 ET=90 HP=90 HT=120	eye=pulse #007FFF	sound=--	1500 ms
3	EP=90 ET=90 HP=90 HT=120	eye=scanner #007FFF	sound=01/004	2000 ms
4	EP=90 ET=90 HP=45 HT=110	eye=scanner #007FFF	sound=--	500 ms

Saved Sequences

greeting4 frames -- 5.0 s

▶↺+✖

look_around8 frames -- 12.0 s

▶↺+✖

alert6 frames -- 6.4 s

▶↺+✖

idle3 frames -- 7.0 s

▶↺+✖

Import JSON

Export

Verify

Sequence-Editor mit Playlist-Steuerung.

Tipp: WebUI aktualisiert Sequence-Liste und Playlist-Status automatisch. Wenn Aenderungen nicht erscheinen: Ctrl+F5 fuer Hard-Reload -- manche Browser cachen zu aggressiv.

Teil II

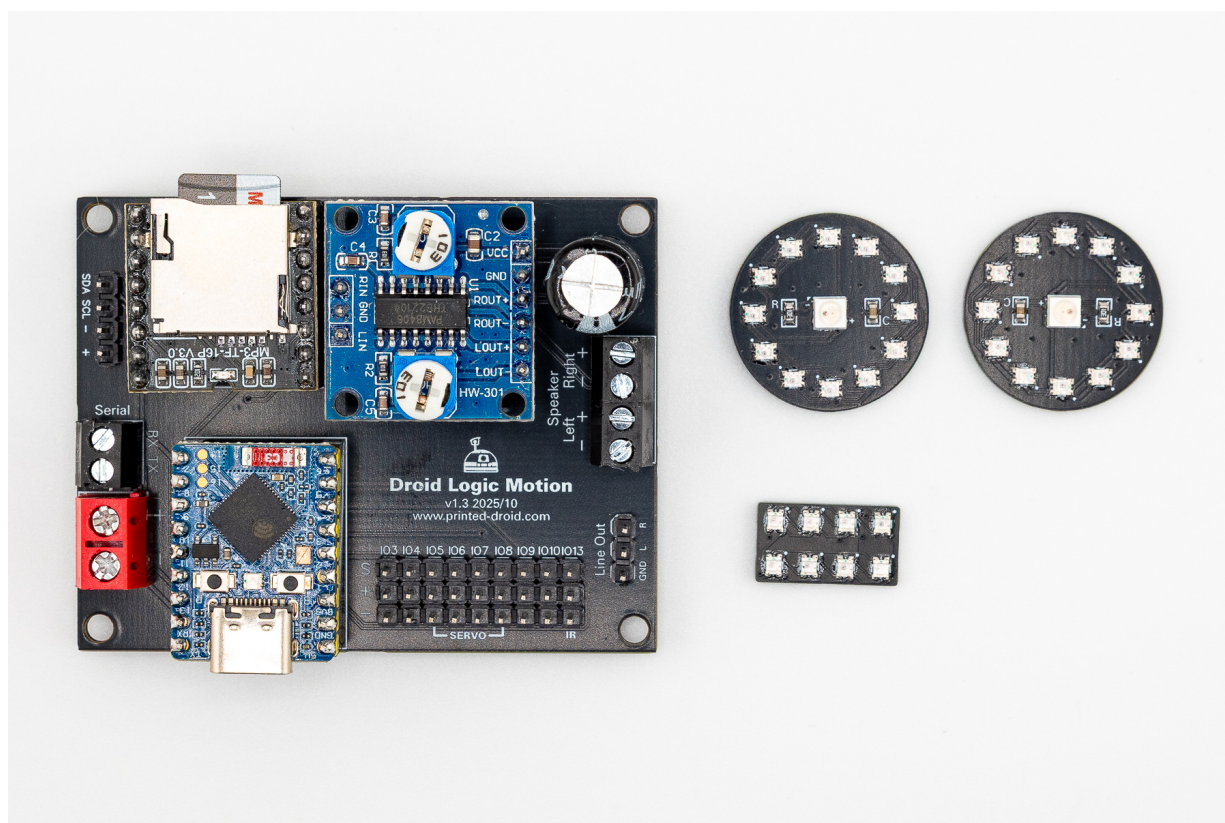
Hardware

Board & Pin-Belegung

Empfohlen ist das **Droid Logic Motion v1.3**, das ein Waveshare ESP32-S3-Zero plus noetige Level-Shifter, Header und DFPlayer-Footprint enthaelt. Standalone ESP32-S3-Zero auf Breakout-Board geht auch -- die Firmware ist egal, solange die GPIO-Belegung unten passt.

GPIO-Pin-Belegung (v1.3.0)

GPIO	Funktion	Richtung	Hinweise
GP1	I2C SDA	I/O	Fuer kuenftige Erweiterung reserviert.
GP2	I2C SCL	OUT	Fuer kuenftige Erweiterung reserviert.
GP3	Left Eye NeoPixel	OUT	7 oder 13 LEDs (zur Laufzeit umschaltbar).
GP4	Right Eye NeoPixel	OUT	7 oder 13 LEDs (zur Laufzeit umschaltbar).
GP5	Eye Pan Servo	PWM	LEDC ch 0.
GP6	Eye Tilt Servo	PWM	LEDC ch 1.
GP7	Head Pan Servo	PWM	LEDC ch 2.
GP8	Head Tilt Servo	PWM	LEDC ch 3.
GP9	IR Receiver	IN	NEC/Sony codecs, learning mode.
GP10	Detail LED Strip	OUT	WS2812, 1-8 LEDs einstellbar.
GP11	DFPlayer TX	OUT	UART2 TX -> DFPlayer RX.
GP12	DFPlayer RX	IN	UART2 RX -< DFPlayer TX.
GP13	unbelegt	--	Seit v1.2.x frei (war in v1.1 Detail-LED).
GP21	Status LED	OUT	Single WS2812.



Droid Logic Motion v1.3 Controller mit Eye-Boards und Detail-LED-Board.

Steckergruppen

Der Silkscreen auf dem Controller beschriftet jede Steckergruppe an Ort und Stelle. Die Tabelle unten ordnet jedes Label seiner elektrischen Funktion zu und -- wo zutreffend -- der GPIO, die ihn bedient.

Silkscreen	Typ	Zweck
SD Card	--	DFPlayer-microSD-Slot.
SDA SCL +/-	4-pin header	I2C-Bus-Terminal (GP1 SDA, GP2 SCL, plus 5V/GND). Fuer kuenftige Erweiterung reserviert.
Serial RX TX	4-pin header	Optionaler Debug-Header. USB-CDC funktioniert ohne.
5V Input	2-pole screw	Haupt-Stromeingang. 5V/3A Minimum, 5V/5A empfohlen.
USBC	USB-C	Programmierung + Strom via ESP32-S3-Zero native USB-CDC.
SERVO (S/+/-)	3 cols x 9 rows	Servo-Header beschriftet IO3 bis IO13. Eye Pan = IO5, Eye Tilt = IO6, Head Pan = IO7, Head Tilt = IO8 in v1.3.0. IO10 fuehrt die Detail-LED-Datenleitung. Andere IOs sind reserviert.
IR	3-pin header	IR-Empfaenger (Signal/+/-) auf GP9.

Silkscreen	Typ	Zweck
Speaker Left/Right +/-	4-pole screw	Verstaerker-Ausgang. Breitband-Lautsprecher. 4-8 Ohm
Line Out (L/R/GND)	3-pin header	Line-Pegel-Ausgang fuer externen Verstaerker. Umgeht den On-Board-PAM8406.

LED-Boards (separat vom Controller)

Eye-Boards (links/rechts): 7-LED- oder 13-LED-Varianten. Jedes Board hat einen 3-Pin-Header (Data In / +5V / GND). *Data In* des linken Auges an GP3, des rechten an GP4. Versorgung und Masse von derselben 5V-Schiene wie der Controller.

Detail-Board: 8 LEDs in 2x4-Layout, 3-Pin-Header. *Data In* an GP10 (IO10-Zeile in der SERVO-Gruppe, Signal-Pin).

Verdrahtungs-Hinweise

- **Gemeinsame Masse:** wenn Servos und ESP32-S3 verschiedene Versorgungen haben: Massen verbinden. Der DFPlayer braucht zwingend gemeinsame Masse mit dem ESP32-S3, sonst ist die UART-Kommunikation unzuverlaessig.
- **Servo-Signal-Level:** ESP32-S3 gibt 3,3V-Logik aus. Die meisten Hobby-Servos akzeptieren das; falls ein Servo nicht reagiert, hilft ein externer Level-Shifter auf 5V.
- **NeoPixel-Signal-Level:** WS2812-Strips wollen offiziell 5V-Logik, funktionieren ueber kurze Kabel (<50 cm) aber meist auch mit 3,3V. Bei laengeren Strecken: 74AHCT125-Level-Shifter am Strip.
- **DFPlayer Einschalt-Reihenfolge:** erst Masse, dann VCC. Hot-Plugging kann den SD-Card-Halter kosten.
- **5V-Schienen-Disziplin:** kurze, dicke Versorgungsleitungen vom Netzteil zum Controller; nicht ueber mehrere lange duenne Adern durchschleifen.
- **GP13 ist FREI seit v1.2.x** -- da nichts anschliessen, ausser du weisst, dass die Firmware auf diesem Pin keine Peripherie sucht.

Tipp: bei einem eigenen Carrier-Board GP1/GP2 (I2C) fuer kuenftige IMU- oder Sensor-Erweiterungen freilassen. GP0, GP43-46 und USB-Pins von Strapping-Konflikten freihalten.

Stromversorgung & Netzteil

Stromversorgung ist die haeufigste Ursache fuer instabiles K-2SO-Verhalten. Brownouts beim DFPlayer-Start oder bei NeoPixel-Flashes zeigen sich als Random-Reboots, verlorenes WiFi, verlorene IR-Codes und stummer DFPlayer. Dieses Kapitel erklart das Budget und die Workarounds.

Strombudget

Komponente	Ruhe	Peak
ESP32-S3-Zero	~80 mA	~250 mA (WiFi TX bursts)
DFPlayer Mini idle	~20 mA	~250 mA (track start)
4x SG90 servos idle	~40 mA	~600 mA pro Servo unter Last
2x 13-LED Eyes white max	je ~80 mA	je ~700 mA
Detail strip 8 LEDs white max	~50 mA	~480 mA
Status LED	vernachlaessigbar	~30 mA
Worst Case Gesamt	--	~3,5 A kurzzeitig, ~1,5 A dauerhaft

5V/3A ist das funktionierende Minimum. Ein 5V/2A-Netzteil brownoutet, wenn der DFPlayer einen Track waehrend eines NeoPixel-Flashs startet. Ein 5V/5A-Netzteil (USB-PD-Trigger-Modul + Step-Down oder gutes Labornetzteil) loest das Problem.

Empfohlene Topologie

Fuer Builds am Kostuem oder im Display:

- **Eine 5V/5A-Schiene** aus gutem Netzteil, 5A-Sicherung, Hauptschalter am Eingang.
- **Servo-Versorgung sternfoermig** zum Netzteil; Servo-Massen nicht ueber das ESP32-Board verketten.
- **USB nur fuer Programmierung** -- wenn die Firmware drauf ist, auf das eigene Netzteil umschalten.

Akku-Builds

Fuer mobile Builds: 2S LiPo (7,4V nominal) in einen 5V/5A-Buck-Konverter. Einzel-Zelle LiPo (3,7V) vermeiden -- bricht unter Servo-Last schnell unter 4V ein. Ein 2S 2200-3000 mAh-Pack reicht fuer 1-2 h moderate Aktivitaet. LVC-Modul (Low-Voltage-Cutoff) zum Zellschutz dazu.

Tipp: der ESP32-S3 gibt "Brownout detector was triggered" vor dem Reset aus. Wenn du das im Serial-Log siehst: das Netzteil ist Verdaechtige Nummer 1, nicht die Firmware.

LED-Hardware

Der Controller treibt vier unabhängige NeoPixel-Ketten: zwei Augenringe, einen Detail-Strip und eine Status-LED. Alle vier nutzen das WS2812-Protokoll auf RMT-Kanaelen (der ESP32-S3 hat 4 RMT-TX-Kanaele, exakt passend). Dieses Kapitel behandelt die Hardware-Varianten und Defaults.

Eye-NeoPixels: 7-LED vs 13-LED

Zwei Eye-Ring-Varianten werden unterstützt und die Wahl ist zur **Laufzeit** umschaltbar, nicht compile-time:

Variante	Layout	Geeignet fuer
7-LED	LEDs 0-6 in einem Ring	Kompakte Augensitze, einfachere Builds.
13-LED (Default)	LED 0 = Mitte, LEDs 1-12 = Aussenring	Iris- und Targeting-Modes nutzen den Center-Pixel; 13-LED gibt allen 14 Modes die volle Wirkung.

Umschalten via Serial-CLI: `led eye 7` oder `led eye 13`. Die Einstellung ist im EEPROM gespeichert. Die Boot-Animation passt sich an.

Detail-LED-Strip

Der Detail-Strip auf GP10 nimmt 1 bis 8 WS2812-LEDs auf. Anzahl ueber `detail count N` einstellen (im EEPROM gespeichert). Muster: solid color, random flicker, auto-mode. Helligkeit 0-255, Default 150.

Status-LED

Eine einzelne WS2812 auf GP21 visualisiert den System-Zustand. Die States haben feste Farben:

State	Farbe	Muster	Bedeutung
BOOT	Blue	Blinken	Initialer Boot, vor WiFi.
WIFI_CONNECTING	Yellow	Blink	STA-Verbindungsversuch laeuft.
WIFI_CONNECTED	Green	Solid 2s	STA up, IP erhalten.
WIFI_DISCONNECTED	Red	Solid	STA verloren.
MODE_SCANNING	Ice blue	Pulse	Scanning-Personality aktiv.
MODE_ALERT	Red	Pulse	Alert-Personality aktiv.
MODE_IDLE	Amber	Pulse	Idle-Personality aktiv.
IR_ACTIVITY	White	Flash	IR-Befehl empfangen.
SERVO_ACTIVITY	Blue	Flash	Servobewegung ausgeloeset.
AUDIO_ACTIVITY	Green	Flash	Audio-Wiedergabe ausgeloeset.

State	Farbe	Muster	Bedeutung
ERROR	Red	Schnell-Blinken	Allgemeiner Fehlerzustand.
LEARNING_MODE	Purple	Blink	IR-Lernmodus laeuft.
CONFIG_MODE	Cyan	Pulse	Setup-Wizard / Monitor-Modus.

Tipp: die Status-LED ist dein erstes Diagnose-Werkzeug. Wenn WiFi nie hochkommt (gelb blinkt 30+ s), startet der AP-Fallback. Wenn die LED dauerhaft blau bleibt, ist die Boot-Sequenz haengen geblieben -- ins Serial-Log schauen.

Teil III

Kernfunktionen

Servo-Kalibrierung

Jeder Build ist mechanisch leicht anders. Kalibrierung setzt die vier Servos so, dass die Augen bei "Center" geradeaus blicken und die Limits die mechanischen Anschlaege deines Builds respektieren. Wer das ueberspringt, kriegt entweder schielende Augen oder ein abgenudeltes Getriebe -- beides schlecht.

Default-Werte

Servo	Center	Min	Max
Eye Pan	90	60	120
Eye Tilt	90	60	120
Head Pan	90	0	180
Head Tilt	90	0	180

Kalibrierungs-Workflow

1. Kopf locker montieren

Der Kopf sollte auf den Servohebeln liegen, aber nicht durch sein Eigengewicht belasten. Finger frei von allen Servoarmen.

2. Erst zentrieren

`servo eye center 90 90` (oder eigenen Wert). Augen sollten geradeaus blicken. Anpassen bis es stimmt, dann `save`.

3. Eye-Limits setzen

Augen nach links pannen bis die Mechanik gerade nicht mehr weich laeuft -- das ist `panMin`. Rechts analog `panMax`. Hoch = `tiltMax`, runter = `tiltMin`. Dann: `servo eye limits panMin panMax tiltMin tiltMax`.

4. Kopf zentrieren

`servo head center 90 90`.

5. Head-Limits setzen

`servo head limits panMin panMax tiltMin tiltMax`. Min muss kleiner als max sein -- die Firmware lehnt `min >= max` mit klarer Fehlermeldung ab.

6. Pruefen

`servo test all` faehrt jedes Servo von Min bis Max. Wenn was klemmt, kratzt oder ueber einen Anschlag will: Limits fixen und neu testen.

7. Als Profil speichern

Wenn alles passt: `profile save MeinBuild`. Bis zu 5 Profile speicherbar.

Min < Max-Pruefung (NEU v1.2.4): die Firmware lehnt Servo-Limit-Befehle ab, bei denen $\text{min} \geq \text{max}$ ist. Früher wurden invertierte Limits stillschweigend akzeptiert, mit erratischer Bewegung als Folge.

Profile

Profile buendeln: Servo-Center und -Limits, Modus, Lautstaerke, Eye-Brightness, Scan/Alert-Timings, Mode-Farben. Fuenf Slots (0-4). `profile list`, `profile save name`, `profile load N`, `profile delete N`, `profile show N`. Speichern wird abgelehnt, wenn alle 5 Slots voll sind -- erst loeschen (NEU v1.2.4-Wache).

Eye-Animationen

Die Eye-Animations-Engine hat 14 Modes. Umschalten mit `led mode <name>` oder ueber die WebUI. Farbe, Helligkeit und Animations-Geschwindigkeit sind vom Modus unabhaengig und gelten fuer den jeweils aktiven Modus.

Animation-Modes

Mode	Beschreibung	Geeignet fuer
<code>solid_color</code>	Statische Farbe, keine Animation.	7 / 13 LED
<code>fade_off</code>	Vom aktuellen Farbwert auf aus blenden.	7 / 13 LED
<code>fade_color</code>	Weicher Fade zwischen zwei Farben.	7 / 13 LED
<code>flicker</code>	Random Helligkeits-Flicker.	7 / 13 LED
<code>pulse</code>	Weicher Helligkeits-Puls.	7 / 13 LED
<code>scanner</code>	Scanner-Sweep ueber alle LEDs.	7 / 13 LED
<code>iris</code>	Ring pulsiert, Center bleibt statisch.	nur 13 LED
<code>targeting</code>	Ring rotiert, Center blinkt.	nur 13 LED
<code>ring_scanner</code>	Scanner-Sweep nur im Ring.	nur 13 LED
<code>spiral</code>	Spirale von aussen nach innen.	7 / 13 LED
<code>focus</code>	Ring blinkt, Center bleibt an.	nur 13 LED
<code>radar</code>	Radar-Sweep im Ring.	nur 13 LED
<code>heartbeat</code>	Synchroner Herzschlag-Puls auf beiden Augen.	7 / 13 LED
<code>alarm</code>	Synchroner Alarm-Flash auf beiden Augen.	7 / 13 LED

Farbe & Helligkeit

`led color R G B` setzt die aktive Farbe (0-255 je Kanal). `led brightness 0-255` setzt die Helligkeit. Die WebUI hat zusaetzlich die Shortcuts `red` / `green` / `blue` / `white` / `off` als Buttons. Helligkeit ist pro Profil speicherbar und uebersteht Reboots.

[Image placeholder] eye_modes_overview

Eye-Animation-Modes (13-LED-Variante).

(Image will be supplied later. The PDF builds without it.)

Tip: Die Modes IRIS, TARGETING, RING_SCANNER, FOCUS und RADAR brauchen den Center-Pixel und sehen auf einem 7-LED-Ring unvollstaendig aus. Auf 7-LED-Hardware: bei den 7-LED-tauglichen Modes bleiben (`solid`, `fade_*`, `flicker`, `pulse`, `scanner`, `spiral`, `heartbeat`, `alarm`).

Audio (DFPlayer Mini)

Audio kommt vom **DFPlayer Mini**, angeschlossen ueber UART (GP11 TX, GP12 RX, 9600 Baud). Er spielt MP3-Files von einer microSD. Das Printed-Droid-Paket enthaelt 40+ K-2SO-Soundclips im Ordner K-2SO Audio Files/.

SD-Card-Vorbereitung

- **Format:** FAT32, max 32 GB.
- **Ordner-Layout:** /01/, /02/, ... -- zweistellige Ordernamen, max 99.
- **Datei-Benennung:** 001.mp3, 002.mp3, ... -- dreistellig, max 999 pro Ordner.
- **Encoding:** Standard-MP3, mono oder stereo, Sample-Rate 8-48 kHz.
- **Hidden Files:** macOS legt `._*`-Resource-Forks an. Mit `dot_clean` entfernen oder unter Linux/Windows mounten und loeschen.

CLI-Referenz

Befehl	Effekt
<code>sound show</code>	Aktuelle Lautstaerke und Audio-Status zeigen.
<code>sound volume 0-30</code>	DFPlayer-Lautstaerke setzen (0=mute, 30=max).
<code>sound play <track></code>	Track aus Root spielen.
<code>sound folder <f> <t></code>	Track <i>t</i> aus Ordner <i>f</i> spielen.
<code>sound stop</code>	Wiedergabe stoppen.

Default-Lautstaerke & Auto-Mode

Default-Lautstaerke beim ersten Boot: 20 (von 30). Die Personality-Engine spielt im Scanning- oder Alert-Modus zufaellige K-2SO-Lines aus einem konfigurierten Ordner, mit einstellbarer Pause (Default 8-20 Sekunden). Pause anpassen mit `timing sound <min> <max>`.

DFPlayer-not-ready-Warnung: wenn die SD fehlt, falsch formatiert ist oder der DFPlayer-Chip die aeltere Klon-Variante ist, erscheint "DFPlayer not ready" im Serial-Log. Erst SD pruefen; wenn die OK ist: anderes DFPlayer-Modul testen.

IR-Fernbedienung einrichten

Der Controller kann jede Infrarot-Fernbedienung mit NEC-, Sony- oder aehnlichem Standard-Codec lernen. Bis zu 21 Buttons sind programmierbar. Nach dem Lernen kann jeder Button auf eine eingebaute Aktion oder (NEU v1.3.0) auf eine gespeicherte Sequenz gemappt werden.

Learning-Mode-Workflow

1. Lernmodus betreten

learn im Serial Monitor. Die Status-LED wechselt auf Lila-Blinken.

2. Button auf der Fernbedienung druecken

Der Controller faengt den IR-Code und fragt nach einem Namen fuer den Button.

3. Button benennen

Kurzes Label wie play, stop, 7. Enter druecken.

4. Fuer jeden Button wiederholen

Jederzeit mit exit oder normal beenden.

5. Pruefen

show listet alle programmierten Buttons mit Codes.

Lernmodus blockiert andere Tasks. Waehrend learn ist, pausieren Servo-Updates und Animationen. Nicht am Kostuem-Droiden lernen.

IR-Befehle

Befehl	Effekt
learn	Lernmodus betreten (lila LED).
scan	Live-IR-Codes ausgeben (Diagnose).
show	Alle konfigurierten Buttons auflisten.
clear	Alle IR-Mappings loeschen (mit Bestaetigung).
ir on / ir off	IR-Empfaenger aktivieren / deaktivieren.

[Image placeholder] ir_remote_layout
Typische 21-Button-Lernfernbedienung.
(Image will be supplied later. The PDF builds without it.)

Tipp: wenn dieselbe physische Fernbedienung fuer denselben Button zwei leicht unterschiedliche Codes sendet (manche tun das mit einem Repeat-Bit), reicht ein einmaliges Lernen. Der Controller behandelt beide als denselben Button.

Personality-Modi

Im Leerlauf laeuft der Droid autonom in einem von drei Personality-Modes. Der Modus steuert, wie oft sich die Augen bewegen, wie weit, wie oft Sounds kommen und welche Ruhefarbe die Augen haben. Umschalten ueber mode <name> oder die WebUI.

Mode	Charakter	Bewegung	Default-Farbe
scanning	Methodisch, beobachtend.	Langsame Eye-Sweeps alle 3-6 s, grosser Winkel.	Ice blue
alert	Schnell, reaktiv.	Schnelle Eye-Darts alle 0,5-1,5 s, kleiner Winkel.	Red
idle	Energiesparend, fast still.	Minimale Bewegung, gelegentliches Zucken.	Amber

Mode-Verhalten anpassen

timing show zeigt aktuelle Werte. Anpassen mit:

Befehl	Range	Effekt
timing scan move <min> <max>	20-200 ms	Scanning-Eye-Move-Dauer-Bereich.
timing scan wait <min> <max>	500-30000 ms	Scanning-Pause zwischen Bewegungen.
timing alert move <min> <max>	5-100 ms	Alert-Eye-Move-Dauer-Bereich.
timing alert wait <min> <max>	100-5000 ms	Alert-Pause zwischen Bewegungen.
timing sound <min> <max>	1000-300000 ms	Zufaelliges Sound-Intervall (beide Modi).

Wake- / Sleep-Verhalten

Nach 60 Minuten Inaktivitaet (kein IR, kein Web-Request, kein Servo-Befehl) geht der Droid in einen Low-Activity-Zustand ("not awake"). Jeder Input weckt ihn wieder. Der Voice-Trigger /trigger/sleep erzwingt Sleep, /trigger/wakeup erzwingt Wake.

Sequence-Recording (NEU v1.3.0)

Die Sequence-Recording-Engine -- neu in v1.3.0 (promoviert aus v1.2.5-Beta) -- nimmt eine Reihe "Frames" (diskrete Posen mit Sound-Cues) auf und spielt sie als weiche Bewegung ab. Sequenzen werden als JSON-Files in LittleFS gespeichert, ueberleben Reboots und lassen sich zu Playlists verketteten (naechstes Kapitel) oder auf IR-Buttons mappen (uebernaechstes).

Was wird pro Frame aufgenommen

- 4x Servo-Positionen (Eye Pan/Tilt, Head Pan/Tilt) zum Zeitpunkt des Frame-Hinzufuegens.
- Eye-Animation-Mode, Farbe, Helligkeit.
- Detail-Strip-Muster, Farbe, Helligkeit.
- Optionaler Sound-Trigger: Folder + Track + Volume.
- Frame-Dauer in Millisekunden (1-60000 ms).

Kapazitaet

Grenze	Wert	Hinweise
Frames pro Sequenz	200	Hard Cap; Aufnahme stoppt bei Erreichen.
Sequenz-Name-Laenge	1-31 chars	Erlaubt: A-Z, a-z, 0-9, Unterstrich, Bindestrich.
Speicher verfuegbar	~1.5-2 MB	Von den 4MB ESP32-S3-Flash, partitioniert fuer LittleFS.
Frame-Groesse	~80 B JSON / ~24 B RAM	JSON-Overhead vs In-Memory-Struct.

Aufnahme-Workflow (Serial-CLI)

1. Aufnahme starten

`seq new "MeinName" --` Name folgt den Regeln oben.

2. Droid posen

Mit WebUI-Gamepad oder beliebigen `servo` / `led` / `detail` / `sound`-Befehlen den gewuenschten Zustand setzen.

3. Frame erfassen

`seq frame <duration_ms> --` z.B. 1500 fuer 1,5 Sekunden.

4. Wiederholen

Posen, erfassen, posen, erfassen. Bis zu 200 Frames.

5. Speichern

`seq save` schreibt das JSON nach `/sequences/<name>.seq` in LittleFS.

6. Jederzeit abbrechen

`seq cancel` verwirft die laufende Aufnahme ohne Speichern.

Wiedergabe-Befehle

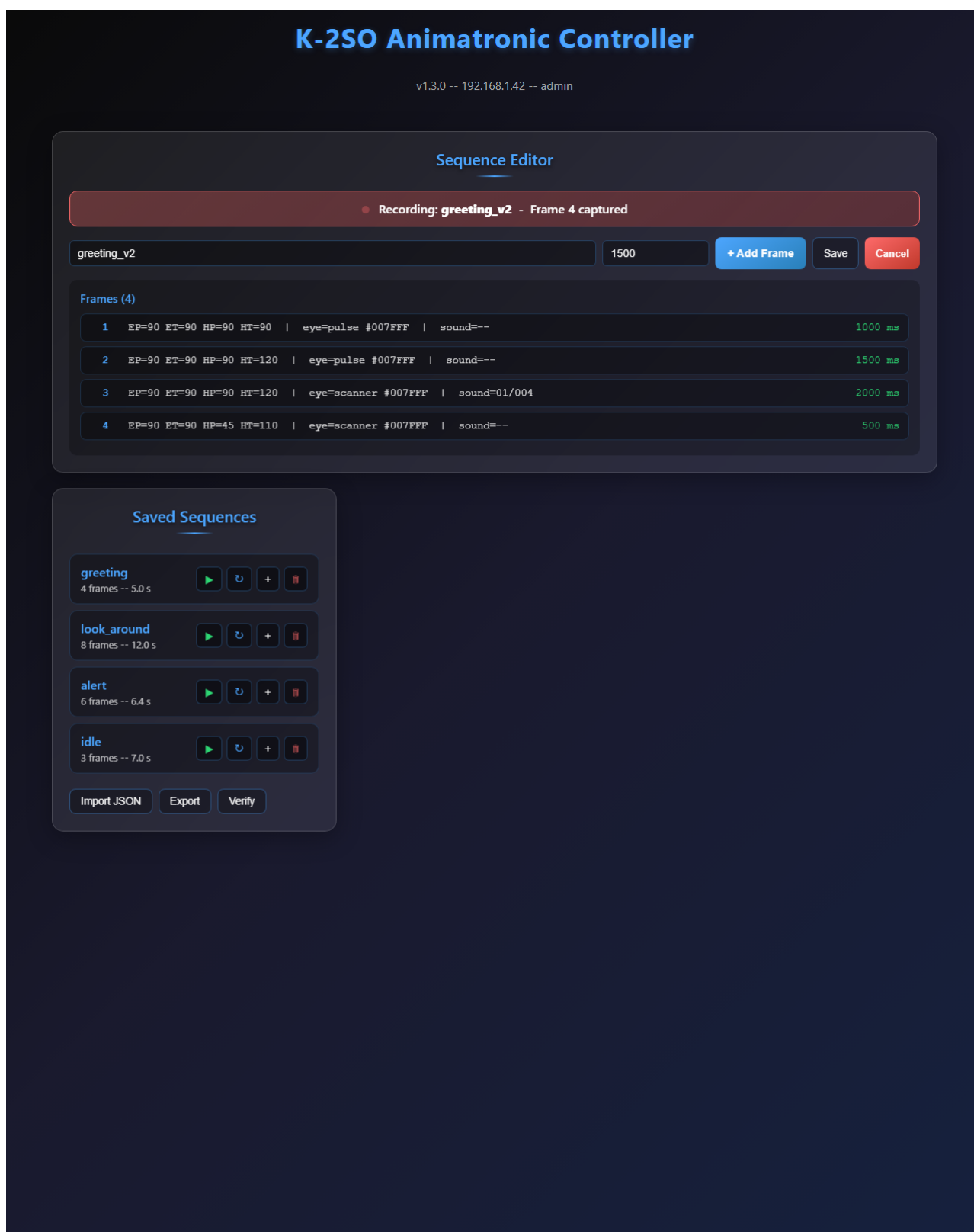
Befehl	Effekt
<code>seq play "name"</code>	Einmal abspielen.
<code>seq loop "name"</code>	Im Loop spielen, bis gestoppt.
<code>seq stop</code>	Aktive Wiedergabe stoppen.
<code>seq list</code>	Alle gespeicherten Sequenzen auflisten (max 20 angezeigt).
<code>seq info "name"</code>	Metadaten: Frame-Anzahl, Gesamtdauer.
<code>seq rename "old" "new"</code>	Sequenz umbenennen.
<code>seq delete "name"</code>	Loeschen mit Bestaetigungsprompt.
<code>seq status</code>	Aktuellen Aufnahme- / Wiedergabe-Zustand zeigen.
<code>seq format</code>	LittleFS neu formatieren (mit Bestaetigung). NEU v1.3.0 -- ersetzt stilles Auto-Format.
<code>seq verify "name"</code>	NEU v1.3.0 Follow-up -- JSON-Integritaet einer gespeicherten Sequenz pruefen.
<code>seq verify all</code>	NEU v1.3.0 -- alle gespeicherten Sequenzen auf einmal pruefen (gut vor einer Show).
<code>seq export "name"</code>	NEU v1.3.0 Follow-up -- Sequenz-JSON auf Serial ausgeben (Backup).
<code>seq import</code>	NEU v1.3.0 Follow-up -- Sequenz-JSON pasten und importieren.
<code>seq duplicate "old" "new"</code>	NEU v1.3.0 -- existierende Sequenz unter neuem Namen kopieren, ohne neu aufzunehmen.
<code>seq stats</code>	NEU v1.3.0 -- LittleFS / Sequenzen / Playlists Uebersicht (freier Platz, Datei-Anzahlen).

Crash-sicheres Speichern (NEU v1.3.0 Follow-up)

Seit dem v1.3.0-Follow-up nutzt seq save einen Temp/Backup/Replace-Ablauf auf LittleFS: der neue Inhalt wird zuerst nach <name>.tmp geschrieben, das existierende Ziel wird zu <name>.bak umbenannt, die Temp-Datei wird nur dann zum eigentlichen Ziel befoerdert, wenn der JSON-Write geklappt hat, und das Backup wird bei jedem Fehler zurueckgespielt. Effekt: du kannst die letzte gute Kopie einer Sequenz nicht durch Ueberschreiben verlieren.

Pause und Resume (nur Web UI)

Pause und Resume gibt es nur in der WebUI (und auf den HTTP-Endpunkten /seq/pause bzw. /seq/resume). Es gibt absichtlich keine seriellen Befehle dafuer -- seriell ist der "exakte Pause-Zeitpunkt" zu schwer zu treffen.



WebUI-Sequence-Editor mit Frame-Timeline.

Tipp zu Dauern: 300-800 ms fuer schnelle Bewegungen, 1000-2000 ms fuer normale, 2000-4000 ms fuer langsame / dramatische, 500-1000 ms fuer Pausen. Mehrere kurze Sequenzen wirken natuerlicher als eine 60-Sekunden-Riesenaufnahme.

Playlist-Chaining (NEU v1.3.0)

Eine Playlist verkettet bis zu 10 gespeicherte Sequenzen und spielt sie nacheinander ab. Damit baust du laengere Performances aus kurzen, wiederverwendbaren Bausteinen: *greeting* -> *look_around* -> *scan* -> *idle*.

Playlist aufbauen

Befehl	Effekt
<code>seq playlist add "name"</code>	An aktuelle Playlist anhaengen (max 10).
<code>seq playlist clear</code>	Playlist leeren.
<code>seq playlist</code>	Aktuelle Playlist zeigen (ohne Argumente).
<code>seq playlist play</code>	Einmal komplett abspielen.
<code>seq playlist loop</code>	Endlos abspielen, bis gestoppt.
<code>seq playlist remove <n></code>	NEU v1.3.0 Follow-up -- Playlist-Eintrag n entfernen (1-basiert).
<code>seq playlist move <from> <to></code>	NEU v1.3.0 Follow-up -- Playlist umsortieren (1-basierte Indizes).
<code>seq playlist save "name"</code>	NEU v1.3.0 -- aktuelle Playlist als benannte .pls-Datei im LittleFS speichern.
<code>seq playlist load "name"</code>	NEU v1.3.0 -- gespeicherte Playlist laden (lehnt fehlende referenzierte Sequenzen ab).
<code>seq playlist list</code>	NEU v1.3.0 -- alle benannten Playlists aus /playlists/ auflisten.

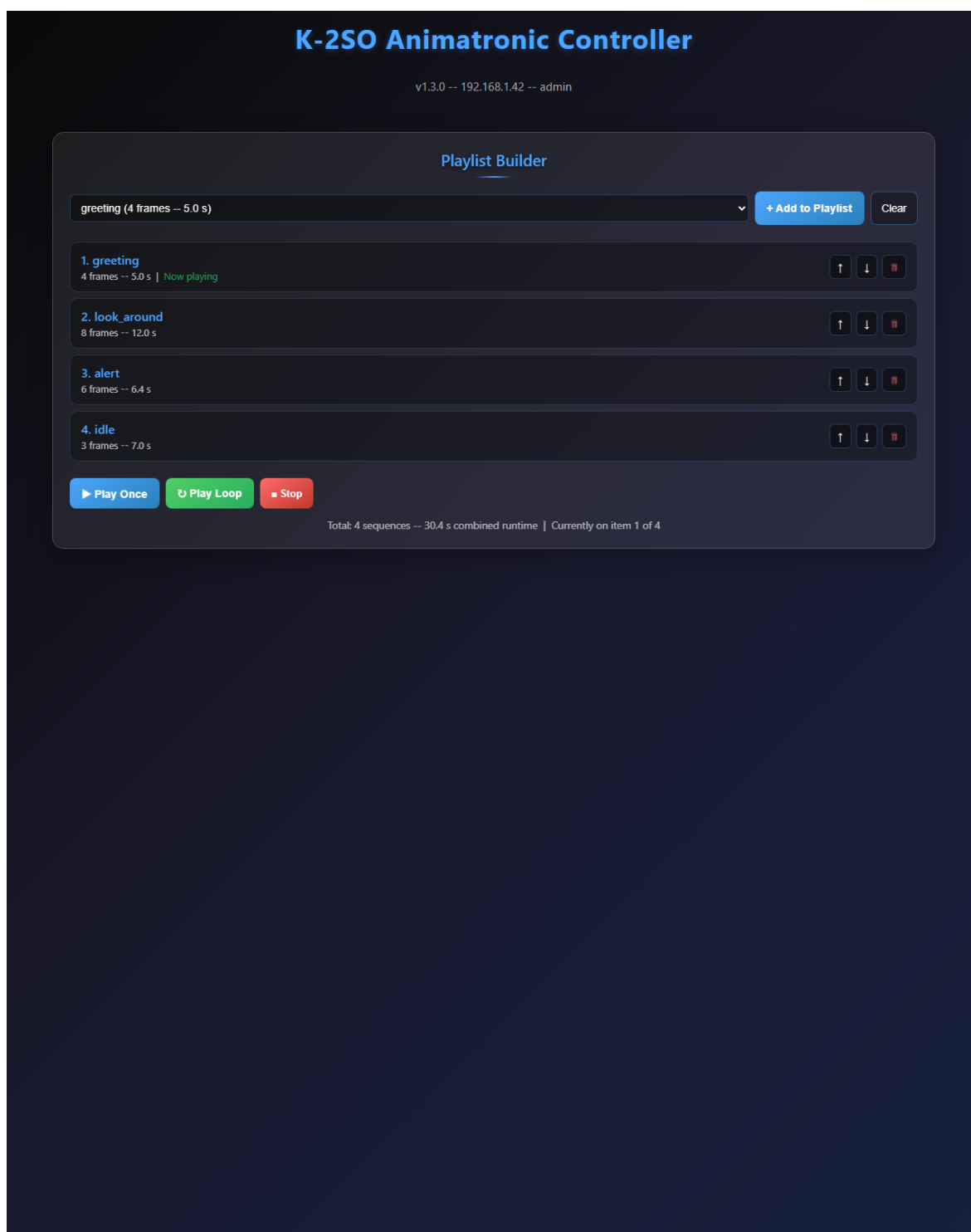
Verhalten und Robustheit

- **Atomische Uebergaenge:** beim Wechsel von einer Sequenz zur naechsten bleibt der Playlist-Zustand erhalten. Frueher (v1.2.5-Beta) brach die Playlist nach dem ersten Uebergang ab -- in v1.3.0 gefixt.
- **Failure-Recovery:** wenn die naechste Sequenz nicht laedt (Datei geloescht, JSON kaputt, Out-of-Memory), stoppt die Playlist sauber und der Droid geht in *Idle* -- kein halb-spielender Zombie-State.
- **Persistente benannte Playlists (NEU v1.3.0):** die aktive Playlist liegt weiter nur im RAM, aber sie laesst sich benannt speichern mit `seq playlist save "meineshow"`. Gespeicherte Playlists liegen unter /playlists/meineshow.pls in LittleFS und ueberleben Power-Cycles. Laden mit `seq playlist load "meineshow"`; das Laden lehnt fehlende referenzierte Sequenzen mit klarer Fehlermeldung ab.

Beispiel-Performance

Eine 30-Sekunden-Performance:

```
seq playlist clear
seq playlist add "greeting" (5 s)
seq playlist add "look_around" (10 s)
seq playlist add "alert" (8 s)
seq playlist add "idle" (7 s)
seq playlist play
```



Playlist-Builder in der Web-UI.

Tipp: kurze, wiederverwendbare Sequenzen sind besser als eine grosse fest verdrahtete Performance. Damit kannst du fuer verschiedene Convention-Szenen Playlists mischen, ohne neu aufzunehmen.

IR-zu-Sequenz-Mapping (NEU v1.3.0)

IR-zu-Sequenz-Mapping (neu in v1.3.0) belegt jeden der 21 gelernten IR-Buttons mit einer gespeicherten Sequenz. Button druecken -> passende Sequenz spielt. Mappings liegen im EEPROM und ueberleben Reboots.

Voraussetzungen

- IR-Fernbedienung ist bereits gelernt (siehe Kapitel IR-Fernbedienung einrichten).
- Mindestens eine gespeicherte Sequenz (siehe Kapitel Sequence-Recording).
- Buttons sollten sinnvolle Namen haben -- Mapping erfolgt ueber Name, nicht Index.

Mapping-Befehle

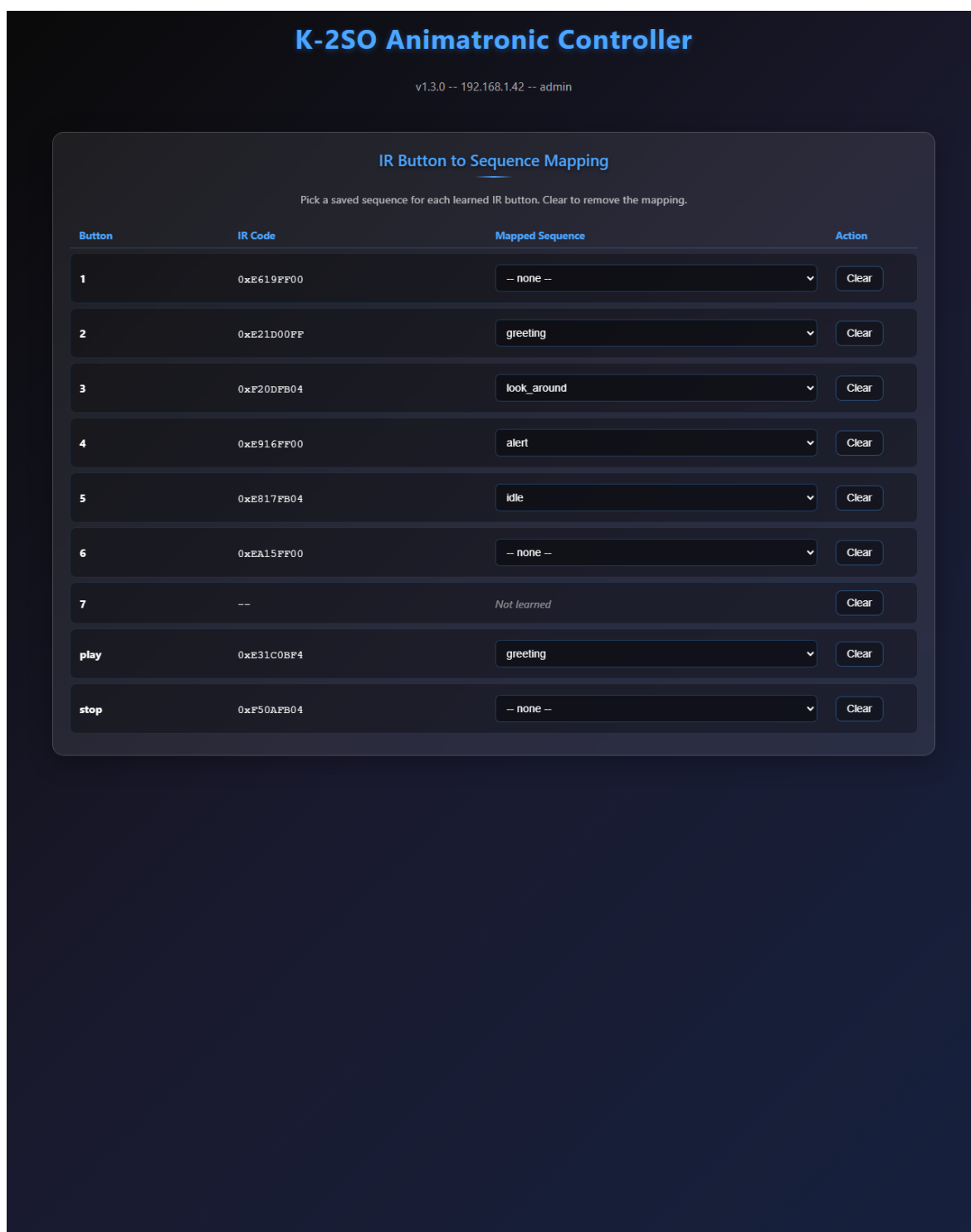
Befehl	Effekt
seq map <button> "name"	Sequenz "name" dem benannten Button zuordnen.
seq map <button> clear	Mapping fuer einen Button entfernen.
seq map	Alle aktuellen Mappings zeigen.

Beispiel

Du hast IR-Buttons 7, 8, 9 gelernt und Sequenzen `greeting`, `alert`, `look_around` aufgenommen:

```
seq map 7 "greeting"  
seq map 8 "alert"  
seq map 9 "look_around"  
seq map # show resulting table
```

Was passiert zur Laufzeit: der Controller dekodiert IR -> findet den passenden Button-Namen -> wenn eine Sequenz gemappt ist, wird sie einmal abgespielt (kein Loop). Im Serial-Log: `IR triggering sequence: greeting`. Wenn keine Sequenz gemappt ist: der Button behaelt seine eingebaute Aktion (oder keine, falls nicht belegt).



IR-zu-Sequenz-Mapping-Tabelle in der WebUI.

Tipp: wenn man den Playlist-Play-Button (z.B. "0") auf eine Platzhalter-Sequenz mappt, startet das nicht die Playlist -- der IR-Trigger spielt immer nur eine einzelne Sequenz. Um eine Playlist per IR zu starten: indirekt ueber eine Voice-Assistant-Routine auf /seq/playlist/play.

Teil IV

Referenz & Anhaenge

Serial-CLI-Referenz

Vollstaendige Referenz fuer die serielle Befehlszeile (115200 Baud). Befehle sind beim Root-Verb case-insensitive und bei Namen (Sequenzen, Profile) case-sensitive. Alle Bestaetigungs-Prompts haben 30 s Timeout (NEU v1.2.4) -- ein steckengebliebener Droid erholt sich von selbst, wenn der Bediener mitten im Prompt weggeht.

Top-Level-Befehle

Befehl	Zweck
help	Befehlsuebersicht ausgeben.
status	Gesamten System-Zustand zeigen.
show	Gelernte IR-Buttons auflisten.
save	Aktuelle Einstellungen in EEPROM speichern.
config	Vollen Config-Dump anzeigen.
default	Config auf Werkseinstellungen zuruecksetzen (mit Bestaetigung).
clear	IR-Mappings loeschen (mit Bestaetigung).
reset	ESP32 neu starten.
monitor	Live-Status-Loop bis Tastendruck.
test	Hardware-Selbsttest-Menue.
demo	Demo-Sequenz starten.
exit / normal	Spezialmodus verlassen (learn / monitor / setup).
backup	Vollstaendige Konfiguration als Hex auf Serial dumpen.
restore	Hex zurueckspielen; Checksum-Pruefung (NEU v1.2.4).
learn	IR-Lernmodus betreten.
scan	Live-IR-Codes ausgeben.
ir on / ir off	IR aktivieren / deaktivieren.
mode <name>	Personality: scanning / alert / idle.

Hierarchische Befehle

Diese haben Subcommands. Verb ohne Argumente eingeben fuer Hilfe.

Gruppe	Subcommands
servo	show / eye center / eye limits / head center / head limits / test
led	show / brightness / color / mode / eye / status / test
detail	show / count / brightness / color / pattern / on / off / auto / test
sound	show / volume / play / folder / stop
timing	show / scan move / scan wait / alert move / alert wait / sound
profile	list / save / load / delete / show
wifi	show / set / reset / reconnect
ap	show / set / reset / enable / disable / start
seq	new / frame / save / cancel / play / loop / stop / list / info / delete / rename / playlist (add / clear / play / loop / remove / move / save / load / list) / map / status / format / verify / verify all / export / import / duplicate / stats

Haeufige Schnell-Befehle

Wofuer	Befehl
Alle Servos zentrieren	<code>servo eye center 90 90 + servo head center 90 90</code>
Hardware-Selbsttest	<code>servo test all</code>
Augen blau, Pulse	<code>led color 0 0 255 + led mode pulse</code>
Sound 1 aus Ordner 1	<code>sound folder 1 1</code>
Aktuelle Einstellungen speichern	<code>save</code>
IP und WiFi-Zustand	<code>wifi show</code>
Config-Backup	<code>backup -- copy the hex output</code>

Voice-Assistants

Der Controller bietet 10 HTTP-Trigger-Endpunkte unter `/trigger/*`. Jeder Voice-Assistant oder jede Home-Automation, die HTTP-GET mit Basic Auth absetzen kann (Google Home, Alexa, Siri Shortcuts, Home Assistant, IFTTT, Node-RED), kann den Droid steuern.

Trigger-Endpunkte

Endpoint	Sprach-Phrase	Effekt
<code>/trigger/wakeup</code>	"K2SO wake up"	Scanning-Modus + blaue Scanner-Augen + Sound.
<code>/trigger/standby</code>	"K2SO standby"	Quiet-Modus, sanftes Blau-Leuchten, keine Bewegung.
<code>/trigger/sleep</code>	"K2SO sleep"	Augen aus, Ton aus, Bewegung aus.
<code>/trigger/demo</code>	"K2SO demo"	Komplette Demo-Sequenz.
<code>/trigger/speak</code>	"K2SO speak"	Zufaellige Voice-Line.
<code>/trigger/alert</code>	"K2SO alert mode"	In Alert-Personality wechseln, amber Augen.
<code>/trigger/scanner</code>	"K2SO scanner eyes"	Klassische Scanner-Augenanimation.
<code>/trigger/alarm</code>	"K2SO alarm"	Rot blinkender Alarm mit Sound.
<code>/trigger/center</code>	"K2SO center"	Alle Servos zentrieren.
<code>/trigger/patrol</code>	"K2SO patrol"	Patrol-Modus mit Bewegungen und Sounds.

Setup-Beispiel: Google Home / IFTTT

1. In IFTTT ein Applet mit Trigger "Google Assistant -- Say a simple phrase". Phrase: *K2SO wake up*.
2. Aktion: "Webhooks -- Make a web request". URL: `http://<deine-droid-ip>/trigger/wakeup`. Methode: GET. Basic-Auth-Header dazu: `Authorization: Basic YWRtaW46azJzbzIwMjU=` (durch eigenes `admin:passwort` base64-kodiert ersetzen).
3. Speichern. Phrase sagen. Der Droid reagiert binnen ~1 s.

Curl / Manueller Test

```
curl -u admin:k2so2025 http://192.168.1.100/trigger/wakeup
```

Authentifizierung ist Pflicht. Alle Trigger-Endpunkte verlangen gueltige HTTP Basic Auth. Wenn `WEB_AUTH_USER` in `config.h` leer ist, ist Auth aus -- nicht empfohlen fuer Produktion.

Konfiguration & Backup

Der Controller speichert seine Konfiguration in einem 2048-Byte EEPROM-Bereich (NVS-backed Flash am ESP32-S3). Das Layout ist identifiziert ueber ein Magic-Byte (0xC0) und eine Versionsnummer. v1.3.0 nutzt Version 2 (mit dem neuen IR-Button-zu-Sequenz-Mapping-Feld); v1.2.x nutzte Version 1. Migration ist automatisch (naechstes Kapitel).

Was wird gespeichert

Gruppe	Inhalt
Identification	magic byte (0xC0), version (2), write counter.
IR Buttons	21 entries: name, IR code, configured flag, sequence-name (NEW v1.3.0).
Servo calibration	Eye Pan/Tilt center+min+max, Head Pan/Tilt center+min+max.
WiFi	STA SSID, STA password, configured flag.
AP	AP SSID, AP password, configured flag, enabled flag.
LEDs	Eye brightness, LED effect speed, eye hardware version (7 / 13 LED), status LED brightness, status LED enabled.
Timing	Scan / Alert / Sound min and max ranges.
Profiles	5 profile slots, each with mode, volume, brightness, servo centers, timings, mode colors.
Integrity	CRC-style checksum at the end.

Backup ueber Serial

backup dumpt die ganzen 2048-Byte-Struct als ASCII-Hex. Output in Textdatei kopieren. Sieht aus wie C0 02 00 00 00 ... 7F -- Bytes durch Leerzeichen getrennt, ~4 KB Text.

Restore ueber Serial

restore fragt nach Bestaetigung, dann gepastetes Hex annehmen. Die Firmware prueft **Magic-Byte + Version + Checksum** vor dem Schreiben. NEU v1.2.4: ungueltige Checksum wird mit klarem Fehler abgelehnt -- frueher wurde Schrott still akzeptiert. **NEU v1.3.0-Follow-up: Dual-Format-Restore.** Sowohl v2-Hex (aktuell) als auch Legacy-v1-Hex (aus v1.2.3- / v1.2.4-Backups) werden direkt akzeptiert. Der Legacy-Pfad nutzt denselben Migrations-Helper wie beim Boot, also landen die Einstellungen ohne Datenverlust im v2-Layout. Multi-Line-Hex-Paste wird korrekt verarbeitet -- Whitespace und Zeilenumbrueche im Block sind ok.

Profile vs Backup

Profile speichern einen Teil der Konfiguration -- Modus, Lautstaerke, Helligkeit, Servo-Center, Timings, Mode-Farben -- in einem von 5 benannten Slots, auf demselben EEPROM. Gedacht fuer schnelle "kleiner / grosser Droid"- oder "scanning / alert"-Presets, nicht als Full-Backup.

Backup/Restore dumpt die ganze 2048-Byte-Struct, inkl. aller Profile, IR-Mappings, WiFi-Credentials etc. Vor grossen Firmware-Updates oder nach sorgfaeltiger Kalibrier-Session sinnvoll.

Das Backup-Hex enthaelt dein WiFi-Passwort im Klartext. Backup-Datei wie eine Passwortdatei behandeln
-- nicht committen, nicht oeffentlich pasten.

Migration v1.2.x auf v1.3.0

v1.3.0 fuehrt eine EEPROM-Layout-Aenderung ein (der IR-Button-Struct ist um 32 Bytes pro Button gewachsen, fuer das Sequence-Name-Feld). Ohne Migration wuerde ein existierender v1.2.3/v1.2.4-Controller beim ersten Boot von v1.3.0 alle Einstellungen verlieren -- WiFi-Passwort, IR-Mappings, Servo-Kalibrierung, Profile, alles. Das war inakzeptabel, also enthaelt v1.3.0 eine **automatische EEPROM-Migration**.

Was passiert beim ersten Boot nach Upgrade

1. Die Firmware liest das EEPROM und prueft Magic-Byte (0xC0) und Version-Feld.
2. Wenn magic = 0xC0 und version = 1: der Migrations-Pfad laeuft. Die Firmware liest das EEPROM im v1-Layout, validiert die v1-Checksum, kopiert alle Felder ins neue v2-Layout, initialisiert die neuen IR-Button-Sequence-Name-Felder leer, rechnet die v2-Checksum neu und schreibt das Ergebnis.
3. Im Serial-Log: Config migrated from v1 (v1.2.3/v1.2.4) to v2 (v1.3.0). All settings preserved.
4. Nachfolgende Boots sehen version = 2 und laufen den normalen Load-Pfad.
5. Wenn die v1-Checksum kaputt war (korrupte Vor-Installation), faellt die Migration auf Factory-Defaults zurueck statt v2 zu korrumpieren.

Erhaltene Felder

- WiFi-STA-SSID und -Passwort.
- AP-SSID, AP-Passwort, AP-Enabled-Flag.
- 21 IR-Button-Mappings (Name + Code + Configured-Flag).
- Servo-Kalibrierung (Eye Pan/Tilt und Head Pan/Tilt Center + Min/Max).
- Alle 5 Profil-Slots (Modus, Lautstaerke, Helligkeit, Servo-Center, Timings, Mode-Farben).
- LED-Einstellungen: Helligkeit, Effekt-Geschwindigkeit, Eye-Hardware-Version (7-LED / 13-LED), Status-LED-Einstellungen.
- Timing-Bereiche (Scan Move/Wait, Alert Move/Wait, Sound-Pausen, Boot-Sequence-Delay).
- Aktueller Personality-Mode und Lautstaerke.

Neue / zurueckgesetzte Felder

- IR-Button-Sequence-Name-Felder sind leer (du belegst sie mit seq map).
- Sequenzen (LittleFS) sind nicht betroffen -- sie leben auf einer separaten Flash-Partition.

Empfohlener Upgrade-Ablauf

1. Auf v1.2.4 ein Serial-Backup ziehen: backup, Hex-Ausgabe sichern.
2. v1.3.0-Firmware ueber USB flashen.
3. Serial Monitor bei 115200 oeffnen; das Boot-Log sollte die Migrations-Meldung zeigen.
4. Pruefen: wifi show, show (IR), servo show, profile list -- alles sollte wie auf v1.2.4 sein.

5. Wenn was falsch ist: v1.2.4 zurueckflashen, Hex-Backup zurueckspielen, neu probieren. Wenn die Migration weiter fehlschlaegt, war das v1-EEPROM schon kaputt -- Factory-Defaults und neu konfigurieren.

Downgrade wird nicht unterstuetzt. Sobald v1.3.0 Version 2 ins EEPROM geschrieben hat, sieht ein zurueckgeflashtes v1.2.4 eine unbekannte Version und faellt auf Defaults zurueck. Falls du zurueckrollen willst: nach v1.2.4-Flash das Serial-Backup zurueckspielen.

Fehlersuche

Die meisten Probleme fallen in wenige Kategorien: Strom, Libraries, WiFi, RMT/IR-Coexistence, SD-Karte. Das Serial-Log bei 115200 Baud ist dein bestes Diagnose-Werkzeug -- die Firmware ist absichtlich gespraechig.

Symptom-zu-Fix-Tabelle

Symptom	Wahrscheinliche Ursache	Fix
Random reboots, "Brownout detector was triggered" in log	Netzteil zu schwach.	5V/3A Minimum, 5V/5A empfohlen. Kurze, dicke Versorgungsleitungen vom Netzteil.
Servos jitter, buzz, do not move smoothly	Brownout, falscher PWM-Kanal oder Limit min >= max.	Erst PSU pruefen. Dann servo show, min < max pruefen. servo test all erneut.
IR commands missed when NeoPixel is animating	Library-Version-Mismatch (NeoPixel + IRremote auf RMT).	Auf getestete Versionen pinnen (siehe Kapitel Sketch hochladen). Beide zusammen aktualisieren.
DFPlayer silent, "DFPlayer not ready" in log	SD-Layout falsch, Hidden-Files oder aelterer Klon-DFPlayer.	FAT32, /01/001.mp3-Layout, dot_clean unter macOS. Anderes DFPlayer-Modul probieren.
WiFi never connects, status LED stays yellow	SSID/Passwort falsch, oder AP-mDNS geblockt.	wifi reset + wifi set IP aus Log nehmen. AP-Fallback nach 30 s.
Web UI 401 Unauthorized	Default-Passwort geaendert oder falsche Creds.	In config.h WEB_AUTH_USER / WEB_AUTH_PASS pruefen.
Sequence loads but plays only one frame, then stops	Truncated JSON-File oder Out-of-Memory.	seq info name zeigt tatsaechliche Frame-Anzahl. Wenn zu klein: neu aufnehmen. Free-Heap sichtbar in /status.
Playlist plays first sequence, then stops	War ein v1.2.5-Beta-Bug -- in v1.3.0 gefixt.	Auf v1.3.0 upgraden. Wenn schon v1.3.0: die naechste Sequenz fehlt oder ist kaputt -- seq list pruefen.
"Invalid sequence name" error	Sequence-Name hat unerlaubte Zeichen.	Erlaubt: A-Z, a-z, 0-9, _, -. Laenge 1-31 Zeichen.
IR learn mode never returns	30 s Timeout erreicht.	exit seriell. Erneut learn. IR innerhalb 30 s senden.
EEPROM seems empty / settings reset themselves	Migrations-Fallback kaputtem v1-EEPROM, NVS-Partition geloescht.	Aus Serial-Backup zurueckspielen. Ohne Backup: Factory-Defaults und einmal neu konfigurieren.

Tipp: im Zweifel monitor. Druckt jede Sekunde Status, bis ein Tastendruck kommt. Auf Free-Heap-Drops, IR-Command-Counter-Aenderungen, Mode-Uebergaenge und Brownout-Indikatoren achten.

Versions-Historie

Dieses Kapitel fasst die Firmware-Versionen zusammen. v1.3.0 ist der aktuelle Stable-Kandidat; v1.2.4 die vorherige Stable; v1.1.0 das Archive. Aeltere Versionen sind nur oberflaechlich dokumentiert -- nicht empfohlen fuer neue Builds.

v1.3.0 -- Promotion-Kandidat (2026-04)

Highlights: promoviert die v1.2.5-Beta-Features zu Stable. Frame-basiertes Sequence-Recording (max 200 Frames), Playlist-Chaining (10 Sequenzen), IR-Button-zu-Sequenz-Mapping. Automatische v1 -> v2 EEPROM-Migration erhaelt alle bestehenden Daten.

Critical-Fixes (gegenueber v1.2.5-Beta): Playlist-Chaining bricht nicht mehr nach erstem Uebergang ab; truncated Sequence-JSON kann Servos nicht mehr mit uninitialisiertem Speicher ansteuern; Sequence-Namen sind gegen Path-Traversal abgesichert; LittleFS formatiert nicht mehr automatisch bei Mount-Failure; JSON-Document-Allokation ist gegen OOM abgesichert.

Quality (gegenueber v1.2.5-Beta): fehlgeschlagene .seq-Writes raechen Teildateien auf; Sequence-HTTP-Endpunkte liefern echte Status-Codes (4xx) bei Fehlern statt HTTP 200 mit "ERROR"-Body; AP-Default-Passwort-Output nutzt das echte Makro statt eines veralteten Literals; restoreFromSerial-Log-Meldung gefixt.

Follow-up-Ergaenzungen: Crash-sicheres Sequence-Save (Temp / Backup / Replace-Flow); Legacy-Serial-Restore akzeptiert jetzt v1-Hex-Backups direkt (Dual-Format-Restore); neue Sequence-Manager-Methoden verifySequence, exportSequenceJson, importSequenceJson, playlistRemove, playlistMove; neue CLI-Befehle seq verify / export / import und seq playlist remove / move; Sequence-/IR-Endpunkte vereinheitlicht auf {ok, message}-JSON-Envelope. WebUI nutzt den neuen Envelope und bietet einen Sequence-Import-Button. getSequenceInfo() loescht bei lokalem OOM keine Datei mehr.

Feature-Round (finale v1.3.0-Ergaenzungen): seq duplicate "alt" "neu" fuer schnelle Varianten ohne neu aufzunehmen; seq verify all fuer Sammelpruefungen vor einer Show; seq stats fuer LittleFS / Sequenzen / Playlists Uebersicht; **persistente benannte Playlists** ueber seq playlist save / load / list (gespeichert als .pls-Dateien in /playlists/); neue HTTP-Endpunkte /seq/duplicate, /seq/verify-all, /seq/stats, /seq/playlist/save, /seq/playlist/load, /seq/playlist/list; /seq/map/list liefert jetzt alle konfigurierten IR-Buttons (gemappt und ungemappt), damit die WebUI leere Zeilen direkt zuweisen kann; WebUI-Buttons fuer *Verify All*, *Stats*, *Copy / Export / Verify* pro Sequenz, *Playlist Save / Load / Move / Remove* und IR-Mapping-Neuzuweisung.

Status: Compile geht durch (~60% Flash (No-OTA-Partition), ~16% RAM). Hardware-Verifikation laeuft.

v1.2.4 -- Vorherige Stable (2026-04)

Highlights: Stabilitaets-Patch auf v1.2.3 mit 6 Fixes aus einem Claude + Codex Cross-Review. EEPROM-kompatibel zu v1.2.3.

Fixes: 2-Sekunden-Main-Loop-Freeze beim WiFi-Reconnect (statusled.cpp); Detail-LED-Random-Loop ohne Iteration-Cap (detailleds.cpp); restoreFromSerial prueft jetzt die Checksum (handlers.cpp); profile save ueberschreibt nicht mehr stillschweigend Slot 0 wenn alle 5 Slots voll sind; Serial-CLI-Confirmations frieren den Droid nicht mehr bei USB-Disconnect ein (5 Stellen + zentraler Helper); Servo-Limits validieren min < max.

v1.2.5 -- Beta (2026-04)

Status: Beta-Quelle fuer die v1.3.0-Promotion. Nicht empfohlen fuer Produktion -- fuer Stable v1.2.4, fuer die neuen Features v1.3.0. Beta-Features (Sequence-Recording, Playlist, IR-Mapping) leben in v1.3.0 mit allen Bug-Fixes.

v1.2.3 -- Stable (2026-03)

Status: vor-vorherige Stable. Hat die 6 Carry-Over-Bugs, die v1.2.4 fixt -- stattdessen v1.2.4 nehmen. Referenz fuer das v1-EEPROM-Layout (von v1.3.0-Migrations-Pfad noch lesbar).

v1.1.0 -- Archive (2026-02)

Status: Erstausgabe, als historische Referenz archiviert. 4-Servo-Motion, einfache Eye-Animation, DFPlayer-Audio, rudimentaere Serial-CLI. Keine WebUI, kein IR-Learning, kein Sequence-Recording. Fuer neue Builds nicht empfohlen -- auf v1.2.4 oder v1.3.0 upgraden.